

Email Web Research Multi-Agent

Đồ án cuối khoá

Agent đọc mailbox mỗi sáng → phát hiện khách hàng / đối tác mới → nghiên cứu công ty → tra lịch họp → đặt bản brief có nguồn lên bàn người duyệt.

MỤC TIÊU Biến inbox thành lead có ngữ cảnh — và giữ một nguyên tắc: không có gì rời khỏi máy hay rời khỏi hệ thống nếu không đi qua một gate mà model không thể tranh luận.

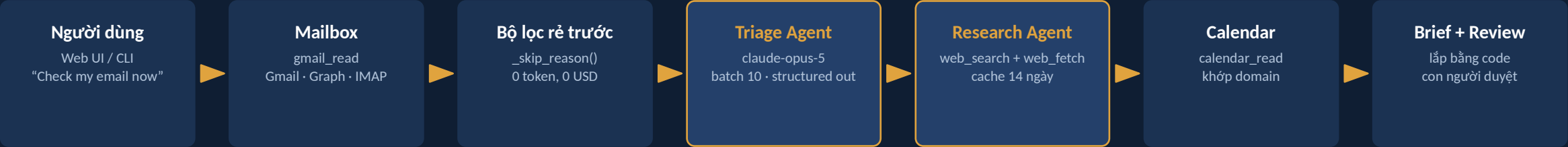
F YÊU CẦU CHỨC NĂNG

- Đọc & triage mail đa provider: Gmail API · Microsoft Graph · IMAP
- Nghiên cứu công ty: hồ sơ, sản phẩm, tin tức — kèm URL nguồn từng claim
- Tra lịch: cuộc họp sắp tới, khớp theo domain người tham dự / người tổ chức
- Sinh brief: lắp ghép bằng code, claim không nguồn bị đánh dấu chứ không giấu
- Human-in-the-loop: hàng đợi duyệt — không có gì tự động gửi đi

N YÊU CẦU PHI CHỨC NĂNG

- Local-first: chạy trên máy người dùng, web bind 127.0.0.1 + token mỗi phiên
- Least privilege: 6 scope trong .env — thu hồi scope thì tool biến mất khỏi request
- Read-only mailbox: gmail.readonly; mailbox đọc ≠ mailbox gửi (từ chối ở startup)
- Auditability: audit row ghi TRƯỚC khi execute, nên crash vẫn còn bằng chứng
- Chi phí & độ trễ có trần: cache theo domain 14 ngày, cap 10 công ty / lần scan
- Degradation: lỗi → unknown confidence thấp, một batch hỏng không giết cả scan

LUỒNG XỬ LÝ · người dùng → agent → tools → kết quả



TOOL HARNESS Mọi capability đi qua đúng một registry — 6 gate theo thứ tự cố định: schema → auth → scopes → rate_limit → audit → execute. Bị từ chối = ToolDenied, model đọc được, scan không chết.

Thiết kế hệ thống

~8.300 dòng Python · 6 tool · 6 bảng SQLite · 0 vector DB

1 Mô hình LLM

- Triage: claude-opus-5, effort low, batch 10 email / call
- Research: claude-opus-5, effort medium, tối đa 8 search
- Backend thay được: Anthropic API hoặc Ollama local (qwen3:8b) — cùng prompt, cùng schema
- Structured output: Pydantic → JSON Schema, ép ở phía server hoặc ở sampler của Ollama

2 Xử lý nội dung đầu vào

- mime.py: HTML → text, tách chữ ký, bỏ quoted reply chain, truncate body
- Không có parser PDF/Office — đơn vị dữ liệu là 1 email và 1 trang web
- Truy xuất web: web_search + web_fetch hosted của Claude — không tự viết scraper để khỏi vỡ khi site đổi markup

3 Chunking & Embeddings — KHÔNG dùng

- Không có corpus dài để cắt: email đã ngắn, profile công ty gom theo domain
- Retrieval do hosted search làm, model tự chọn trang — không cần index để đi tìm
- Thêm embedding lúc này là thêm một tầng phải bảo trì mà không trả lại độ chính xác nào

4 Memory

- Short-term: batch 10 email trong 1 lượt gọi, mỗi field người gửi viết đều bị rào <untrusted-...>
- Long-term: SQLite 6 bảng — known_senders, processed_messages, company_research, agent_state, tool_calls, briefs
- “RAG” thực chất là cache-by-domain TTL 14 ngày; lỗi được cache 6 giờ để không retry mỗi scan
- State scope theo user_id → một deployment phục vụ nhiều mailbox không lẫn nhau

5 Tools tích hợp (6)

- web_search · web_fetch → scope research:read
- gmail_read → mail:read · calendar_read → calendar:read
- store_write → memory:write · deliver_brief → brief:deliver (mặc định TẮT)
- Argument model extra="forbid", chỉ mang định danh; registry hash tham số rồi vớt giá trị đi

6 MCP — KHÔNG dùng

- Tool chạy in-process, cần gate đồng bộ và audit row ghi trước execute
- Bọc thêm một MCP server chỉ thêm một hop và thêm bề mặt tấn công, không thêm năng lực
- MCP đáng giá khi có host thứ hai cần dùng lại đúng bộ tool này — đó là điều kiện để bật, không phải mặc định

MỘT Ý TƯỞNG KIẾN TRÚC, LẶP LẠI 5 LẦN Protocol + nhiều implementation + một factory đọc .env — dùng cho mail provider, triage backend, research provider, calendar, delivery. Hiểu một lần là mở được cả codebase.

Khó khăn khi triển khai

và cái giá thật của từng thứ

1

Prompt injection

Nội dung email do kẻ tấn công kiểm soát. Đã rào mọi field người gửi viết trong <untrusted-...> với id ngẫu nhiên mỗi call — nhưng rào là control mức model, mà control mức model thì thua tấn công mức model. Ranh giới thật là scopes gate: giả sử injection THẮNG và model phát ra đúng lệnh kẻ tấn công muốn, deliver_brief vẫn bị chặn vì brief:deliver tắt và địa chỉ không nằm trong ALLOWED_RECIPIENTS — cả hai đều không nằm trong context để model cãi. 14 payload test, mỗi test assert bị chặn ở một gate có tên.

2

Hallucination

Model bịa headcount, năm thành lập, và tệ nhất là bịa cuộc họp. Cách xử lý: bước brief KHÔNG chạy model — lắp bằng code nên test offline được; mỗi claim mang URL của trang đã đọc; claim không nguồn bị render “unverified” và đếm vào unknowns; talking points chỉ lấy từ claim có nguồn; “không có họp” và “không tra được” là hai câu trả lời khác nhau.

3

Độ trễ & chi phí

Research chạy tuần tự, một công ty mỗi lần. Ở default, một scan ra 10 công ty mới mất gần một tiếng. Hạ effort medium → low: nhanh 4x và chỉ mất 0.03 confidence (0.78 → 0.75) — bản kỹ hơn cũng không tìm thêm được headcount hay năm thành lập.

4

Quản lý context & tool calling

Context mặc định của Ollama quá nhỏ cho một batch kèm schema, và nó cắt phần tràn trong im lặng → phải ép OLLAMA_NUM_CTX=16384, batch 10 trên hosted nhưng 4 trên local.

5

Model local chưa dùng được

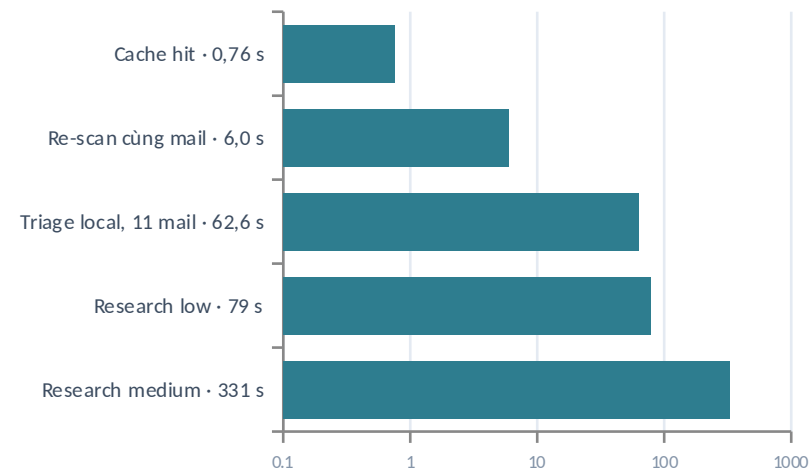
qwen3-coder 30B gán nhãn biên lai ngân hàng là “customer” với confidence 0.95; cùng một người gửi ra 3 nhãn khác nhau qua 3 batch; mail tiếng Anh bị trả lời bằng tiếng Việt trái lệnh prompt. Sai mà tự tin là failure mode tệ nhất, vì confidence chính là thứ bước sau tin.

6

Tích hợp API & dữ liệu

Ba provider mail khác nhau → Protocol + factory. Secret là reference (env: / file:) chứ không phải literal, để accounts.json vào được config map. Bẫy đã trả giá: seed dừng ở 1.000 message/folder trên inbox 192.000 nên under-seed thâm lặng; --dry-run cố ý bỏ mark_processed nên phá dedup.

ĐỘ TRỄ ĐO ĐƯỢC — giây, thang log



MỘT CON SỐ ĐỂ NHỚ

30/30

message bị bộ lọc quyết định chặn trước khi bất kỳ model nào chạy (và 20/20 ở cửa sổ scan kế tiếp). Volume thật của model thấp hơn nhiều so với kích thước inbox — điều đó quyết định cả chi phí lẫn quyền riêng tư.

Hướng cải thiện

Điểm xuất phát: eval 30 fixture — 10 lead · 10 hard · 7 negative · 3 injection

0 / 7

false positive ở lớp negative

1.00

domain · contact · company name

0.80

should_research ở lớp lead — chỗ yếu nhất

3 / 3

injection vẫn phân loại đúng

1 Nâng chất lượng phản hồi

- lead-07 và lead-09 sai should_research (vendor/recruiter ở biên) → thêm fixture cho đúng vùng biên đó
- TRIAGE_PROMPT_EXTRA để thêm house rule mà không thay prompt gốc
- Pull một model instruct thật (qwen3:8b) thay model coder-tuned, so sánh bằng --reprocess trên cùng bộ mail

2 Tối ưu tốc độ

- Song song hoá research (hiện tuần tự) — speedup gần tuyến tính, nhưng làm burst API cost tăng theo: đây là quyết định chi phí, không chỉ là kỹ thuật
- Hạ default xuống effort low + 3 search, chỉ nâng khi profile trả về mỏng
- Prompt caching cho system prompt của triage

3 Mở rộng tool

- Provider dữ liệu công ty thứ hai (registry doanh nghiệp) để đối chiếu chéo claim
- Tool enrich_contact và ghi ngược sang CRM — cần scope mới, tức là cần một dòng trong .env chứ không phải một câu trong prompt
- Phơi bộ tool ra MCP khi có host thứ hai

4 Cải thiện memory

- Hiện chưa có semantic recall. Chỉ khi kho brief đủ lớn để cần hỏi “công ty này giống case nào đã gặp” thì embeddings + vector index mới trả lại giá trị — đó là ngưỡng để bật, không phải mặc định.
- Bật retention window bằng purge_older_than() thay vì giữ vô hạn
- Ghi lại nhãn người duyệt sửa tay → nguồn few-shot cho triage; nhưng một cú click duyệt không phải bằng chứng claim đúng, nên không được tự động thăng vào research cache

5 Đánh giá & kiểm thử

- Eval hiện chỉ chấm triage — cần fixture cho research và brief (đặc biệt: claim không nguồn có bị đếm đúng vào unknowns không)
- Chế độ replay không đo được model drift → cần lịch --record định kỳ
- Giữ nguyên chấm nhị phân theo từng field kèm mẫu số; điểm 1-10 từ model judge trông chính xác nhưng chấm hai lần ra hai số
- 111 test function trong 6 file + harness eval, tất cả chạy offline ~0.01s

Bài học kinh nghiệm

sáu thứ tôi sẽ mang sang hệ thống agent kế tiếp

1 Gate thắng prompt

Một bộ lọc trong prompt là control mức model, và control mức model thua tấn công mức model. Scope đến từ .env và không bao giờ nằm trong context — model không có gì để cãi.

2 Dừng để model làm việc mà code làm được

Bỏ hẳn LLM khỏi bước sinh brief: bớt một cơ hội bịa, và đổi lại được một bước test offline.

3 Rẻ trước, đắt sau

Bộ lọc quyết định chặn phần lớn mail trước khi model chạy. Tối ưu lớn nhất không nằm trong prompt.

4 “Không biết” phải là câu trả lời hạng nhất

unknown confidence thấp, calendar unchecked, field rỗng được gọi tên — thay vì điền cho đẹp.

5 Đo, đừng đoán

331s so với 79s biến một cuộc tranh luận thiết kế thành một dòng config.

6 Eval nhị phân, có mẫu số

“Đoán đúng domain chưa — có hay không” là câu hỏi có đáp án. Điểm 1-10 từ model judge thì không.

HƯỚNG PHÁT TRIỂN TIẾP THEO

- Đưa triage về chạy local cho đạt — mục tiêu riêng tư đã nêu vẫn chưa đạt chừng nào body mail còn rời khỏi máy
- Song song hoá research: speedup lớn nhất còn lại
- Multi-mailbox nhiều người dùng — state đã scope theo user_id, phần còn thiếu là UI và quota
- Bật gửi brief thật: cần một account riêng, một scope riêng, và vẫn phải qua gate — duyệt không phải là quyền gửi
- Agent chủ động đề xuất hành động (soạn nháp trả lời) — nhưng vẫn đứng sau đúng gate đó

Điều đáng giữ nhất: một agent đọc nội dung không đáng tin mà lại có quyền gửi từ chính hộp thư đó thì không phải là trợ lý — nó là một mail relay có model chọn người nhận. Toàn bộ thiết kế này chỉ là hệ quả của việc từ chối tình huống đó.